Program

```
#include <stdio.h>
int main(){
    int n;
    scanf("%d", &n);
    int i=1;
    int s=1;
    int count=3
    while(s<=n){
        count+1;
        i++;
        count++;
        s+=i;
        count++;
    }
    printf("%d", count);
    return 0;
}
```

output:

Input	Expected	Got	
9	12	12	✓
4	9	9	✓

Problem 1: Finding Complexity using counter Method

Aim:

To convert the given algorithms into a C program and determine the time complexity using counter method.

Algorithm:

1. Start the program.
2. Read the positive integer n
3. Initialize i = 1, s = 1 count = 3
4. Repeat while s<=n;
   - Increment count
   - Increment i
   - Increment count
   - Add i to s
   - Increment count
5. Print the value of count.
6. Stop the program.

Result:-

The program's time complexity to verified using counter method.

Program

```
#include <stdio.h>
int main(){
    int n;
    int count = 1;
    scanf("%d", &n);
    if (n == 1) {
        count++;
    }
    else {
        count++;
        for (int i = 1; i <= n; i++){
            count++;
            count++;
            count++;
            break;
        }
        count++;
    }
    printf("%d", count);
    return 0;
}
```

output:

Input	Expected	Got
2	12	12 ✓
1000	5002	5002 ✓
143	717	717 ✓

Program 2 Finding complexity using counter Method.

Aim:
To convert the given algorithm into a C program and determine the time complexity using the counter method.

Algorithm.

1. Start the program
2. Declare integer variables n and count
3. Read the value of n
4. Initialize count = 0
5. If n == 1
   - Increment count
   - Execute the outer loop from i = 1 to n

   For each iteration
   - Increament count
   - Execute the inner loop
   - Increment count three times

   void the loop.
   - Break the inner loop after its

first iteration
   - Increment count.
   - Increment count after the outer loop

7. print count
8. Stop the program.

Result:-

The program's time complexity verified using counter method.

Program

```
#include <stdio.h>
int main(){
    int n;
    scanf("%d", &n);
    int count = 0;
    for (int i = 1; i<=n; i++){
        count++;
        count++;
        if (n%i==0){
          count++
        }
    } count++;
   printf("%d", count);
}
```

output

Input	Expected	Got	
12	31	31	✓
25	54	54	✓
4	12	12	✓

Program 3 : Finding complexity using counter method

Aim

To convert the given algorithm into a program and determine the time complexity of it using counter method.

Algorithm

1. Start the program.
2. Read the integer n.
3. Initialize count = 0
4. Repeat the loop for i = 1 to n
   - Increament count
   - Increament count
   - check whether n % i == 0
   - If true, increament count
5. After the loop, increment count
6. print the value of count.
7. Stop the program.

Result:

The program's time complexity is verified using counter method.

Program

```
#include <studio.h>
int main(){
    int n;
    int c=1;
    scanf("%d", &n);
    for(int i=n/2; i<n; i++){
        c++;
        c++;
        for(int j=1; j<n; j++){
            c++;
            c++;
            for(int k=1; k<n; k++){
                c++;
                c++;
            }
        }
    }
    printf("%d", c);
```

Output

Input	Expected	Got
4	30	30
10	212	213

Program 4 : Finding Complexity using counter method :

Aim:

To convert the given program into c program and determine the time complexity.

Algorithm

1. Start
2. Read the integer n:
3. Initialize counter, c = 1
4. Execute the outer loop
5. For each iteration of outer loop
   - Increment c
   - Execute third loop and increment c
   - Increment c
7. Increment c
8. Print the value of c
9. Stop the program.

Result :-

The program's time complexity is verified using counting method.

Program

```
#include <stdio.h>
int main(){
    int n;
    scanf("%d", &n);
    int count = 1;
    int rev = 0;
    count++;
    int remainder;
    count++;
    while(n!=0){
        count++;
        remainder = n%10;
        count++;
        rev = rev*10 + remainder;
        count++;
        n/=10;
        count++;
    }
    printf("%d", count);
}
```

output

Input	Expected	crot	
12	11	11	✓
1234	19	19	✓

Program 5: Finding complexity using counter method.

AIM:-

To find the time complexity and counter variable for a given algorithm.

Algorithm:

```
void reverse (int n)
{
  int rev = 0, remainder;
  while (n != 0)
  {
    remainder = n % 10;
    rev = rev * 10 + remainder;
    n /= 10;
  }
  printf (rev);
}
```

Time complexity:

O(log n) -- (single while loop)

RESULT:

Thus the time complexity and counter variable are found for the given algorithm using counter method.

Input	Expected	Got
49	9	5

1-G coin problem

AIM:-

To write a program to take value V and we want to make change for V Rs

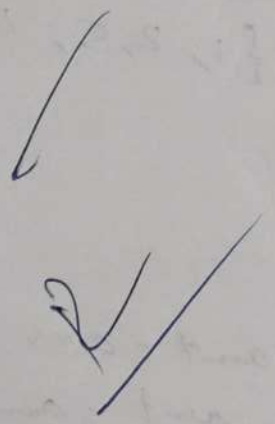
Algorithm:-

```
#include <stdio.h>
int main(){
    int amt;
    scanf("%d",&amt);
    int arr[9]={1, 2, 5, 10, 20, 50, 100, 500,
        1000};
    int i=8;
    int count=0;
        if(amt-arr[i]>=0){
            amt=amt-arr[i];
            count++
        }
        i--;
    }
    printf("%d", count);
    return 0;
}
```

RESULT:-

Hence the program is verified.

Input	Expected Output
2	2
1 2	2
3	2
1 2 3	



2 - G cookie

AIM :-

To maximize the number of your content children and output the maximum number.

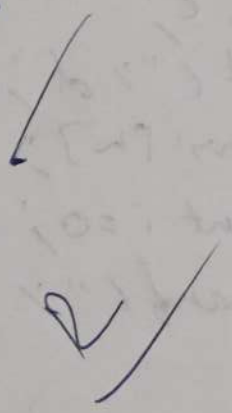
Algorithm :-

```
#include <stdio.h>
int main() {
    int n, c;
    scanf("%d", &n);
    int arr1[n];
    for (int i=0; i<n; i++) {
        scanf("%d", &arr1[i]);
    }
    scanf("%d", &c);
    int arr2[c];
    for (int i=0; i<c; i++) {
        scanf("%d", &arr2[i]);
    }
}
    printf("%d", count);
}
```

RESULT :-

Hence, the proved program is verified.

Test	Input	Expected	Got	
Test case 1	3 132	18	18	✓
		389	389	✓
Test case 2	4 7496			



3- G. Burger problem

AIM:-

To print the minimum number of kilometers to run to burn out the calories.

Algorithm:-

```
#include <stdio.h>
#include <math.h>
int main()
    int n;
    scanf("%d", &n);
    int arr[n];
    for(int i=0; i<n; i++){
        scanf("%d", &arr[i]);
    }
  }
}
 int sum=0
 for(int i=0; i<n; i++){
     sum = sum + pow(2,i)*arr[i];
 }
 printf("%d", sum);
}
```

RESULT:

Hence, the program has been verified.

OUTPUT:-

=

Input	Expected	Got
5 2 5 3 4 0	40	40

✓

4 G - Array sum - max problem

AIM:-

To print maximum Array sum to be printed.

Algorithm:-

```
#include <stdio.h>
#include <math.h>
int main (){
    int n;
    scanf("%d", &n);
    int arr[n];
    for (int i=0; i<n; i++){
        for (int j=i+1; j<n; j++){
            if (arr[i]>arr[j]){
                temp = arr[j];
                arr[i] = arr[j];
                arr[j] = temp;
            }
        }
    }
    int sum=0;
    for (int=0; i<n; i++){
        sum = sum + arr[i];
    }
    printf("%d", sum);
}
```

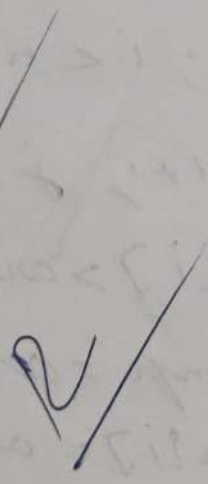
RESULT:-

Hence the program is verified.

Output:-

=

Input	Expected	Got
3	28	28
1		
2		
3		
4		
5		
6		



5- Q product of Array elements
- Minimum

AIM:-

To rearrange the sum of product of pairs

ALGORITHM:-

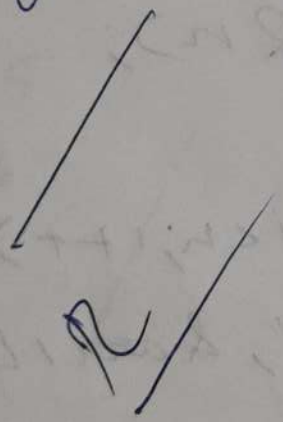
```
#include <stdio.h>
int main (){
    int n;
    scanf("%d", &n);
    int arr[n];
    for (int i=0; i<n; i++){
        scanf("%d", &arr1[i]);
    }
int arr2[n];
for(int i=0; i<n, i++){
    scanf("%d", &arr2[i]);
}
int temp
int sum=0
for(int i=0; i<n; i++){
    sum = sum + arr1[i] * arr2[i];
}
printf("%d", sum);
}
```

RESULT:-
Here the program is verified

OUTPUT:-

=

Input	Expected	Got
5	2	2
1		
1		
1		
0		
0		



AIM:-

= To check the number of zeros present in the given array.

Algorithm:-

=

```
#include <stdio.h>
int firstzero (int arr[], int low, int high)
. if (low >= high) %n 2
        return high+1;
    }
    int mid = (high + low)/2;
    if (arr[mid] == 0) {
        if mid == 0 || arr (mid-1] == 1
            return mid;
        }
        return first zero (arr, low, mid-1);
    }
    else {
        return first zero (arr, mid+1, high);
    }
}
int main()
    int n;
    scanf("%d", &n)
```

```
}
int fz = firstzero(arr, 0, n-1);
int zerocount = n-fz;
printf("%d", zerocount);
return 0;
}
```

RESULT:-

Hence the program has passed through all test cases.

OUTPUT:-

Input	Expected output	Got
3	3	3
3 2 3		



2 Majority element A

AIM:-

To get the majority element is the element that appears more.

Algorithm:-

```
#include <stdio.h>
int majority (int a[], int left, int right
{
    if (left == right)
        return a[left];
    int mid = (left + right)/2;
    int mid = (leftmajority, mid, a);
    int rightmajority = majority (a, mid+1, right)
    if (leftmajority == rightmajority)
        return leftmajority;
    int leftcount = 0, rightcount = 0;
    for (int i = left; i <= right; i++)
    {
        if (a[i] == leftMajority)
            leftcount++;
```

```
if (a[i] == right majority)
        right count++;

else return right majority;

int a[n];
for (int i = 0; i < n; i++)
        scanf("%d", &a[i]);
        printf("%d", majority(a, 0, n-1));
}
```

RESULT:

Hence the program has passed through all the test cases.

```
if (a[i] == right majority)
        right count++;

else
    return right majority;

int a[n];
for (int i = 0; i < n; i++)
        scanf("%d", &a[i]);
    printf("%d", majority(a, 0, n-1));
}
```

RESULT: ✓

Hence the program has passed through all the test cases.

OUTPUT:-

=

Input	Expected	Got
6	2	2
1		
2		
8		
10		
12		
19		
5		



3-Finding Floor Value

AIM:-
= To get the first line that contains Integer and float value for x.

Algorithm:
=

```
#include <stdio.h>
int floorvalue(int a[], int low, int high)
{
  if (low > high)
    return -1;
  int mid = (low + high)/2;
  if (a[mid] == x)
    return a[mid];
  if (a[mid] >= x)
    return floorValue(a, low, mid-1, x);
  return result;
}
int main()
{ int n, x;
  Scanf("%d", &n);
  int a[n];
  for (int i=0; i<n; i++)
    Scanf("%d", &a[i]);
```

```
scanf("%d", &x);
printf("%d", floorvalue(a, 0, n-1));
return 0;
}
```

RESULT:

Hence the program has passed all test cases.

Output :-

Input	Expected output	Crct
4	4	4
2	10	10
4		
8		
10		
14		

4 - Two. Elements sum to x

AIM:-

To print the first line containing integers recursive program using divide & conquer.

Algorithm:-

```
#include <stdio.h>

int findpair (int arr[], int l, int r, int
 if (l >= r)
     return 0;
 int mid = (l+r)/2;
 if (find pair (arr, l, mid, x))
  return 1;
 if (find pair (arr, l, x))
  return 1;
 for (int i = 1 ; i <= mid; i++) {
    for (int j = mid + 1 ; j <= r; j++) {
    printf ("%d" %d", arr[i], arr[j]);
    return 1;
 }
}
int main (){
   int n, x
  Scanf ("%d", &n);
```

```
    if (!findPair(arr, 0, n-1, x))
        printf("No");

    return 0;
}
```

Result:-

Hence the program has passed all test cases.

OUTPUT :-

=

Input	Expected	Got
5 67 34 12 98 78	12346778 98	



Implementation of Quicksort

AIM:-
= To print all the sorted lists of elements.

Algorithm:-
=

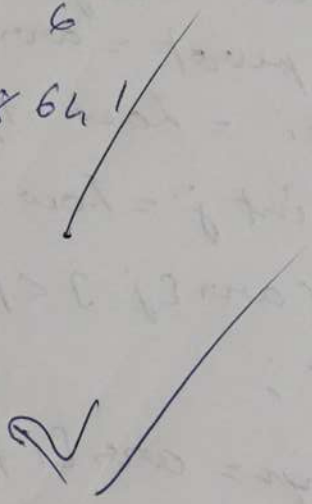
```
#include<stdio.h>
void quicksort(int arr[], int low, int high){
    if(low < high){
      int pivot = arr[high];
      int i = low-1;
    for(int j = low; j<high; j++){
      if(arr[j]<pivot){
      i++;
    int temp = arr[i+1];
    arr[i+1] = arr[high];
    arr[high] = temp;

int main(){
    int n;
     scanf("%d",&n);
     int arr[n];
     for(int i=0; i<n; i++)
        scanf("%d",&arr[i]);
   for(int i=0; i<n;i++);

  return 0;
```

OUTPUT:-

Input	Output	root
6	6	6
25	8641	8641



1 DP playing with Numbers

AIM:-
To print the number of positive ways in' can be represented using 1 & 3.

Algorithm:-

```
#include<stdio.h>
int main(){
    int n;
    scanf("%d", &n);
    long long dp[n+1];
    dp[0]=1;
    for(int i=1; i<=n; i++)
        dp[i]=dp[i-1];

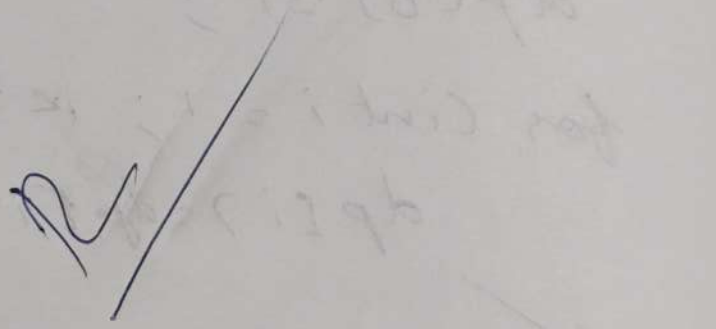
    printf("%lld", dp[n]);
    return 0;
}
```

RESULT:-
Hence the program has passed all test cases

OUTPUT:-

=

Input	Expected	Got
3 1 2 4 2 3 4 8 7 1	19	19



2 - DP Playing with chessboard

AIM:-

To create a program to play the game of chess.

Algorithm:-

```
#include<stdio.h>
int main(){
    int n;
    scanf("%d", &n);
    int a[n][n];
    int dp[n][n];
    for(int i=0; i<n; i++){
        for(int j=0; j<n; j++){
            scanf("%d", &a[i][j]);
        }
    }
    dp[0][0] = a[0][0];
    for(int j=1; j<n; j++){
        dp[0][j] = dp[0][j-1] + a[0][j];
    }
    for(int i=1; i<n; i++){
        for(int j=1; j<n; j++){
            if(dp[i-1][j] > dp[i][j-1]
                else
                dp[i][j] = dp[i][j-1] + a[i][j]
    {printf("%d", dp[n-1][n-1]);
    return 0;
}
```

RESULT:-

Hence the result is verified

OUTPUT:-

Input	Expected	Got
aab	2	2
azb		



3. DP - Longest Common Subsequence

AIM :-

To find the length of common Longest subsequence between two.

Algorithm :-

```
#include <stdio.h>
#include <string.h>

int main(){
    char s1[100], s2[100];
    scanf("%s", s1);
    scanf("%s", s2);

    int n = strlen(s1);
    int m = strlen(s2);

    int dp[n+1][m+1];
    for(int i=0; i<=n; i++)
        dp[i][0]=0;

    for(int i=1; i<=n; i++){
        for(int j=1; j<=m; j++){
            if(s1[i-1]==s2[j-1])
                dp[i][j]=dp[i-1][j-1]+1;
        }
    }
    printf("%d", dp[m][n]);
    return 0;
}
```

RESULT:-

Hence the result is verified

OUTPUT:-

Input	Expected	Got
aab	2	2
azb		



3. DP - Longest Common Subsequence

AIM :-

To find the length of common Longest subsequence between two.

Algorithm :-

```
#include <stdio.h>
#include <string.h>

int main(){
    char s1[100], s2[100];
    scanf("%s", s1);
    scanf("%s", s2);

    int n = strlen(s1);
    int m = strlen(s2);

    int dp[n+1][m+1];
    for(int i=0; i<=n; i++)
        dp[i][0]=0;

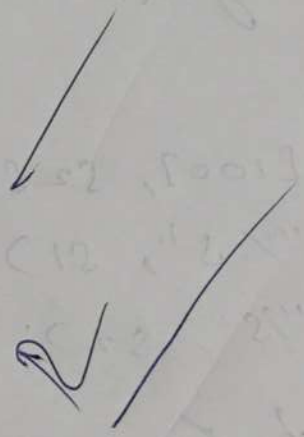
    for(int i=1; i<=n; i++){
        for(int j=1; j<=m; j++){
            if(s1[i-1]==s2[j-1])
                dp[i][j]=dp[i-1][j-1]+1;
        }
    }
    printf("%d", dp[m][n]);
    return 0;
}
```

RESULT:-

Hence the result is verified.

OUTPUT:-

Input	Expected
9	6
-1 3 4 5 2 2 2 3	



AIM:

subseq

Algor

4- DP- Longest non-decreasing Subsequence

AIM:-

To find the length the Non-decreasing subsequence in a given sequence.

Algorithms:-

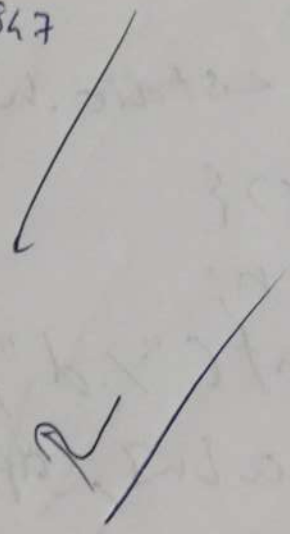
```
#include <stdio.h>
int main(){
    int n;
    scanf("%d", &n);
    int a[n], dp[n];
    for (int i=0; i<n; i++){
        scanf("%d", &a[i]);
        dp[i]=1;
    }
    if (dp[i] > ans)
        ans = dp[i];
}
printf("%d", ans);
return 0;
}
```

RESULT:-

Hence the program has passed all test cases.

Output:-

Input	Expected	Got
11	7	7
10 9 7 6 5 1 2 3 8 4 7		



1 - Finding Duplicates - $O(n^2)$

AIM:-

To find one number that repeats in the array.

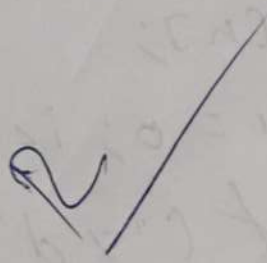
Algorithm:-

```
#include <stdio.h>
int main(){
    int n;
    scanf("%d", &n);
    int arr[n];
    for(int i = 0; i<n; i++){
        scanf("%d", &arr[i]);
    }
        for(int j=i+1; j<n; j++){
            if(arr[i]==arr[j]){
                printf("%d", arr[i];
            }
        }
    }
    return 0;
}
```

RESULT:- Hence the program has passed thro all test cases.

OUTPUT:-

Input	Output	Crot
11		
10 9 7 6 5 1 2 3 8 4 7	7	7



2- Finding Duplicates - O(n) Time complexity, O(1) Space complexity.

AIM:-

To find numbers that repeat in only array of n intgers between 1 and n.

Algorithm:-

```
#include <stdio.h>
int main(){
    int n;
    Scanf("%d", &n);
    for(int i = 0; i<n; i++){
        Scanf("%d", &arr[i]);
    }
    int visarr[100]={0};
    for(int i=0; i<n; i++){
        if(visarr[arr[i]]==0)
            visarr[arr[i]] = 1;
        else
            printf("%d", arr[i]);
    }
    return 0;
}
```

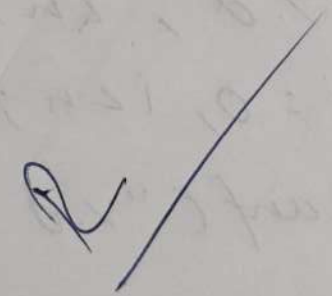
RESULT:-

Hence the program passed all test cases.

OUTPUT:-

=

Input	Output
1	10 57
3 10 1757	
6	
2 7 10 15 57 246	



3 - Print Intersection of 2 sorted arrays
- O(m*n) Time complexity, O(1) Space complexity

AIM:-

To find the intersection of two sorted arrays. To find all elements which occur in both the arrays.

Algorithm:-

```
#include <stdio.h>
int main() {
    int T;
    scanf("%d", &T);

    while (T--) {
        int n1, n2,
        int a[n1];

        for (int i=0; i<n1; i++)
            scanf("%d", &a[i]);

        Scanf("%d", &n2);
    for (int i=0; i<n2; i++)
        Scanf("%d", &b[i]);

    for (int i=0; i<n1; i++){
```

```
if (int j = 0; j < n2 ; j++){
    if (a[i] == b[j]){
        printf("%d", a[j]);
        break;
    }
}
}
printf("\n");
}
return 0;
}
```

RESULT:-

Hence the program is completed and verified.

OUTPUT:-

Input	Expected	Got
3 1 3 5 4	1	1



5- pair with difference - O(n^2) time completely, O(1) space complexity

AIM:-

= To find if there exists 2 indices i and j such that A[j] - A[i] = k - i! = j

Algorithm:-

```
#include <stdio.h>
int main (){
    int n, k;
  scanf ("%.d", &n);
  int A[n];
  for (int i = 0; i < n; i++)
        scanf ("%.d", &A[i]);
       scanf ("%.d", &k);
for (int i = 0; i < n; i++){
     for (int j = i+1 ; j < n ; j++){
          if (A[j] - A[j] == k){
               printf ("1");
              return 0;
          }
    }
}
printf("0")
return 0)
}
```

RESULT:-

= Hence the program has passed all testcases

6 - Pair with Difference - O(n) Time complexity, O(1) Space complexity

AIM:-

To find if there exists two indices i and j such that A[j] - A[i] = k, i != j

Algorithm:-

```c
#include <stdio.h>

int main (){
    int n, k;
    scanf ("%d", &n);
    int A[n];
    for (int i = 0; i < n; i++);
        Scanf ("%d", &A[i];
    Scanf ("%d", &k);

int i = 0, j = 1
while (i < n && j < n) {
    if ( i == j) {
        j++;
    }
    else if (A[j] - A[i] == k) {
        printf ("1");
        return 0;
    }
```

OUTPUT:-

=

Input	output	Crot
3	1	1
1 3 5		



```
else if (A[j] - A[i] == k) {
        printf("1");
        return 0;
    }
    else if (A[j] - A[i] < k){
        j++;
    }
    else {
        i++;
    }
}
printf("0");
return 0;
}
```

RESULT :-

Hence the program has passed all test cases.